NumPy: Numerical Python

Chapter 4: Introduction to Data and Programming with Python

Class 9/10: Artificial Intelligence

Modules, Packages and Libraries

October 27, 2026

What We Will Learn

- What NumPy is and why we need it
- Installing and importing NumPy
- Creating arrays: `array`, `arange`, `ones`, `zeros`, `full`, `random`
- Array attributes: `ndim`, `shape`, `size`, `dtype`
- Methods: `max`, `min`, `sum`, `sort`, and `axis`
- Bonus: `reshape`

Our learning pattern

1. Learn the idea
2. Try the practice questions
3. Check the answers

What is NumPy?

- **Num**erical **Py**thon: a free, open-source package
- Works with **large sets of numbers** quickly
- Its main tool is the **n-dimensional array**
- Pandas, Matplotlib and Scikit-learn are built on NumPy
- First step to becoming a Python data scientist

[Image placeholder]
NumPy logo

Why Do We Need NumPy?

Aspect	NumPy array	Python list
Data type	Same type (homogeneous)	Mixed types allowed
Memory	Uses less memory	Uses more memory
Maths operators	Work directly on the array	Must be applied element by element
Main use	Mathematical operations	Data management

Installing NumPy

Python from python.org does *not* include NumPy

Open the terminal or command prompt and type:

```
pip install numpy
```

- You only need to do this **once** per computer
- Pandas is installed the same way: `pip install pandas`

Importing NumPy

```
import numpy              # whole package
import numpy as np        # short name: np
from numpy import array # only array
from numpy import array as ary
```

Best practice

Use `import numpy as np`, the standard everywhere.

Remember

Every example in this presentation assumes `import numpy as np` has been run first.

What is an Array?

- An ordered collection of values of the **same type**
- Index numbers start from **0**
- 1-D array = **vector**
- 2-D array = **matrix**

1-D (vector)

10	20	30	40
0	1	2	3

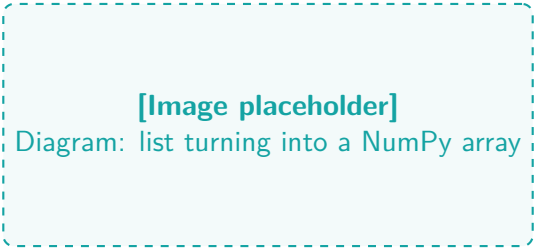
2-D (matrix)

1	2	3
4	5	6

Section 1: Creating Arrays

Five ways to build an array

`array` `arange` `ones / zeros` `full` `random`



Creating Arrays: `np.array()`

Purpose

Converts a Python **list** into a NumPy array. Use a list of lists for a 2-D array.

Example 1: 1-D array

```
a = np.array([1, 2, 3])
print(a)
```

```
[1 2 3]
```

Example 2: 2-D array

```
b = np.array([[1, 2], [3, 4]])
print(b)
```

```
[[1 2]
 [3 4]]
```

Practice Question: `np.array()`

Question 1

```
x = np.array([2, 4, 6])
print(x * 3)
```

Your prediction: _____

Question 2

```
y = np.array([1, 2, 3])
print(type(y))
```

Your prediction: _____

Answer: `np.array()`

Answer 1

```
[ 6  12  18]
```

Operators work **element-wise**:
$2 \times 3,\ 4 \times 3,\ 6 \times 3$.

Answer 2

```
<class 'numpy.ndarray'>
```

The array's type is `ndarray`, an n-dimensional array.

Creating Arrays: `np.arange()`

Purpose

Makes a sequence of numbers: `np.arange(start, stop, step)`. The **stop** value is *not* included.

Example 1: with a step

```
a = np.arange(10, 101, 10)
print(a)
```

```
[ 10  20  30  40  50  60  70  80  90 1(
```

Example 2: only stop

```
b = np.arange(5)
print(b)
```

```
[0 1 2 3 4]
```

Practice Question: `np.arange()`

Question 1

```
print(np.arange(3, 16, 3))
```

Your prediction: _____

Question 2

```
print(np.arange(2, 9))
```

Your prediction: _____

Answer: `np.arange()`

Answer 1

```
[ 3  6  9 12 15]
```

Start at 3, add 3 each time, stop before 16.

Answer 2

```
[2 3 4 5 6 7 8]
```

The default step is 1, and 9 is **not** included.

Creating Arrays: `np.ones()` and `np.zeros()`

Purpose

Create an array filled with all **1s** or all **0s**. Give the shape as a tuple: `(rows, columns)`.

Example 1: ones

```
print(np.ones((2, 3)))
```

```
[[1. 1. 1.]
 [1. 1. 1.]]
```

Example 2: zeros

```
print(np.zeros((2, 2)))
```

```
[[0. 0.]
 [0. 0.]]
```

The dots (`1.`) show the numbers are decimals (floats) by default.

Practice Question: `np.ones()` and `np.zeros()`

Question 1

```
print(np.zeros((2, 3)))
```

Your prediction: _____

Question 2

```
print(np.ones((1, 4)))
```

Your prediction: _____

Answer: `np.ones()` and `np.zeros()`

Answer 1

```
[[0. 0. 0.]
 [0. 0. 0.]]
```

2 rows and 3 columns of zeros.

Answer 2

```
[[1. 1. 1. 1.]]
```

1 row and 4 columns, still printed with double brackets because it is 2-D.

Creating Arrays: `np.full()`

Purpose

Fills an array of a given shape with **one constant value**: `np.full(shape, value)`.

Example 1

```
print(np.full((2, 3), 7))
```

```
[[7 7 7]
 [7 7 7]]
```

Example 2

```
print(np.full((3, 2), 9))
```

```
[[9 9]
 [9 9]
 [9 9]]
```

Practice Question: `np.full()`

Question 1

```
print(np.full((2, 2), 8))
```

Your prediction: _____

Question 2

```
print(np.full((1, 3), 4) + 1)
```

Your prediction: _____

Answer: `np.full()`

Answer 1

```
[[8 8]
 [8 8]]
```

A 2×2 array where every cell is 8.

Answer 2

```
[[5 5 5]]
```

First we get `[[4 4 4]]`. Then $+1$ is added to **every** element.

Creating Arrays: `np.random`

Purpose

Creates arrays of **random** numbers. `random(n)` gives *n* decimals between 0 and 1. `randint(high, size=...)` gives whole numbers below `high`.

Example 1: random decimals

```
print(np.random.random(3))
```

```
[0.62341534 0.18732091 0.93408216]
```

Example 2: random integers

```
print(np.random.randint(10, size=(2, 3)
```

```
[[3 7 1]
 [9 0 4]]
```

Sample outputs only! Your numbers will be different each time you run the code.

Practice Question: `np.random`

Question 1

```
r = np.random.random(6)
print(len(r))
```

Your prediction: _____

Question 2

```
r = np.random.randint(10, size=5)
print(r.max() < 10)
```

Your prediction: _____

Hint: the answer is True or False.

Answer: `np.random`

Answer 1

```
6
```

`random(6)` always makes exactly 6 numbers, even though their values change.

Answer 2

```
True
```

`randint(10, ...)` only gives numbers from 0 to 9, so the maximum is always below 10.

Section 2: Array Attributes and Methods

Learning about our arrays

`ndim` `shape` `size` `dtype` `max/min` `sum` `sort` `reshape`

[Image placeholder]
Picture: a detective's magnifying glass over an array

Attribute: ndim

Purpose

Tells us the **number of dimensions** (axes) of an array. 1-D = vector, 2-D = matrix.

Example 1: 1-D

```
a = np.array([1, 2, 3])
print(a.ndim)
```

```
1
```

Example 2: 2-D

```
b = np.array([[1, 2, 3],
              [4, 5, 6]])
print(b.ndim)
```

```
2
```

Practice Question: `ndim`

Question 1

```
x = np.array([10, 20, 30, 40])
print(x.ndim)
```

Your prediction: _____

Question 2

```
y = np.array([[7, 8, 9]])
print(y.ndim)
```

Your prediction: _____

Answer: `ndim`

Answer 1

```
1
```

One pair of square brackets means a 1-D array.

Answer 2

```
2
```

Double brackets mean 2-D, even though there is only one row.

Attribute: shape

Purpose

Gives the **length of each dimension** as a tuple: `(rows, columns)` for 2-D.

Example 1: 1-D

```
a = np.array([1, 2, 3, 4])
print(a.shape)
```

```
(4,)
```

Example 2: 2-D

```
b = np.array([[1, 2, 3],
              [4, 5, 6]])
print(b.shape)
```

```
(2, 3)
```

Practice Question: shape

Question 1

```
x = np.array([[1, 2],
              [3, 4],
              [5, 6]])
print(x.shape)
```

Your prediction: _____

Question 2

```
y = np.array([9, 8, 7])
print(y.shape)
```

Your prediction: _____

Answer: shape

Answer 1

```
(3, 2)
```

3 rows and 2 columns.

Answer 2

```
(3,)
```

A 1-D array has just one number. The comma makes it a tuple.

Attribute: `size`

Purpose

Gives the **total number of elements** in the array: rows $\times$ columns for 2-D.

Example 1

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])
print(a.size)
```

```
6
```

Example 2

```
b = np.zeros((3, 4))
print(b.size)
```

```
12
```

Practice Question: `size`

Question 1

```
x = np.arange(1, 11)
print(x.size)
```

Your prediction: _____

Question 2

```
y = np.ones((5, 3))
print(y.size)
```

Your prediction: _____

Answer: `size`

Answer 1

```
10
```

`arange(1, 11)` makes 1, 2, . . . , 10, which is 10 numbers.

Answer 2

```
15
```

$5 \times 3 = 15$ elements.

Attribute: dtype

Purpose

Shows the **data type** of the elements. Every element in a NumPy array has the same type.

Example 1: whole numbers

```
a = np.array([1, 2, 3])
print(a.dtype)
```

```
int64
```

Example 2: decimals

```
b = np.array([1.5, 2.5])
print(b.dtype)
```

```
float64
```

On some computers (older Windows) integers may show as `int32`.

Practice Question: dtype

Question 1

```
x = np.arange(5)
print(x.dtype)
```

Your prediction: _____

Question 2

```
y = np.array([1, 2.5, 3])
print(y.dtype)
```

Your prediction: _____

Answer: `dtype`

Answer 1

```
int64
```

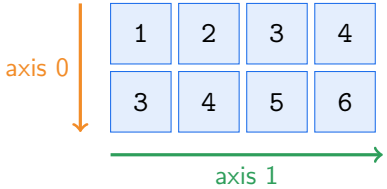
`arange(5)` gives whole numbers, so the type is integer.

Answer 2

```
float64
```

One decimal makes NumPy convert **all** elements to float, so the array stays homogeneous.

Understanding `axis`



- **axis=0**: go **down**, giving one answer per **column**
- **axis=1**: go **across**, giving one answer per **row**
- No axis: use **all** elements

Methods: `max()` and `min()`

Purpose

Find the **largest** or **smallest** value, overall or along an `axis`.

```
ARR = np.array([[1, 2, 3, 4],
                [3, 4, 5, 6]])
```

Example 1: whole array

```
print(ARR.max())
print(ARR.min())
```

```
6
1
```

Example 2: with axis

```
print(ARR.max(axis=0))
print(ARR.min(axis=1))
```

```
[3 4 5 6]
[1 3]
```

Practice Question: `max()` and `min()`

```
M = np.array([[2, 8, 5],
              [9, 1, 7]])
```

Question 1

```
print(M.max(axis=0))
```

Your prediction: _____

Question 2

```
print(M.min(axis=1))
```

Your prediction: _____

Answer: `max()` and `min()`

Answer 1

```
[9 8 7]
```

axis=0, column-wise: max(2,9)=9, max(8,1)=8, max(5,7)=7.

Answer 2

```
[2 1]
```

axis=1, row-wise: min(2,8,5)=2, min(9,1,7)=1.

Method: sum()

Purpose

Adds up the elements: all of them, or column-wise (axis=0) or row-wise (axis=1).

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])
```

Example 1: total

```
print(a.sum())
```

```
21
```

Example 2: with axis

```
print(a.sum(axis=0))
print(a.sum(axis=1))
```

```
[5 7 9]
[ 6 15]
```

Practice Question: sum()

```
N = np.array([[2, 4],
              [6, 8],
              [1, 3]])
```

Question 1

```
print(N.sum(axis=0))
```

Your prediction: _____

Question 2

```
print(N.sum(axis=1))
```

Your prediction: _____

Answer: `sum()`

Answer 1

```
[ 9  15]
```

Column 0: $2 + 6 + 1 = 9$. Column 1: $4 + 8 + 3 = 15$.

Answer 2

```
[ 6  14   4]
```

Row sums: $2 + 4 = 6$, $6 + 8 = 14$, $1 + 3 = 4$. One answer per row.

Function: np.sort()

Purpose

Returns a **sorted copy** (smallest to largest). For 2-D arrays each **row** is sorted. The original array is not changed.

Example 1: 1-D

```
a = np.array([5, 2, 9, 1])
print(np.sort(a))
```

```
[1 2 5 9]
```

Example 2: 2-D

```
b = np.array([[9, 3, 6],
              [8, 2, 7]])
print(np.sort(b))
```

```
[[3 6 9]
 [2 7 8]]
```

Practice Question: `np.sort()`

Question 1

```
x = np.array([40, 10, 30, 20])
print(np.sort(x))
```

Your prediction: _____

Question 2

```
y = np.array([[4, 1, 3],
              [9, 7, 8]])
print(np.sort(y))
```

Your prediction: _____

Answer: `np.sort()`

Answer 1

```
[10  20  30  40]
```

Ascending order, smallest first.

Answer 2

```
[[1  3  4]
 [7  8  9]]
```

Each row is sorted on its own.

Bonus Method: `reshape()`

Purpose

Changes the **shape** of an array without changing its data. The total number of elements must stay the same.

Example 1

```
a = np.arange(6)
print(a.reshape(2, 3))
```

```
[[0 1 2]
 [3 4 5]]
```

Example 2

```
b = np.arange(1, 9)
print(b.reshape(4, 2))
```

```
[[1 2]
 [3 4]
 [5 6]
 [7 8]]
```

Practice Question: `reshape()`

Question 1

```
x = np.arange(8)
print(x.reshape(2, 4))
```

Your prediction: _____

Question 2

```
y = np.arange(10)
print(y.reshape(3, 3))
```

Your prediction: _____

Hint: does it work?

Answer: `reshape()`

Answer 1

```
[[0 1 2 3]
 [4 5 6 7]]
```

$2 \times 4 = 8$ elements, which matches.

Answer 2

```
ValueError: cannot reshape array
of size 10 into shape (3,3)
```

$3 \times 3 = 9$, but the array has 10 elements. The sizes must match.

Cheat Sheet

Code	What it gives	For ARR (2×4)
`ARR.ndim`	number of dimensions	2
`ARR.shape`	(rows, columns)	(2, 4)
`ARR.size`	total elements	8
`ARR.dtype`	data type	int64
`ARR.max()`	largest value	6
`ARR.sum(axis=1)`	row sums	[10 18]
`np.sort(ARR)`	each row sorted	same order here

Thank You! Homework Time

Try these assignments

1. Create NumPy arrays using different methods
2. Get elements using index values
3. Find the shape and reshape an array
4. Search and sort a NumPy array

[Image placeholder]
Celebration / well done image